Behavioural WAL & Assumptions — Why We Built It This Way

`docs/behavioural_wal.md`, `docs/prepayment_wal.md`, and `docs/persistence_structural_split_wal.md` walk through *how* we compute a WAL. This is the companion piece — *why* we made the calls we made, looked at from four angles: the maths that makes each approach correct, the financial risk each one manages, the software design that makes it maintainable, and the business strategy each one actually serves.

Decision: three WAL types, not one

Mathematically — weighted average life is only a trivial calculation (read the maturity date) when a maturity date exists *and* is informative. The moment either condition breaks — no maturity date, or a maturity date that doesn't predict actual cash flow timing — WAL becomes an estimation problem, not a lookup. Forcing one formula across all three cases means either overfitting a formula that assumes a maturity date to products that don't have one, or building unnecessary complexity into the one case that's genuinely simple. This is also why the Basel Committee's own IRRBB standard treats non-maturity deposit repricing and loan prepayment as two distinct disclosure requirements rather than one (Basel Committee on Banking Supervision [BCBS368], 2016) — regulators arrived at the same three-way split for the same underlying reason.

Financially — this is a mispricing question before it's anything else. Pricing a call account as pure overnight money understates its true value as stable funding; pricing a home loan at its full contractual term overstates how long that funding is actually needed. Either error compounds across a whole balance sheet.

In the software — we route on one field, `WAL_TYPE`, decided once per account from a reference sheet rather than re-derived inline wherever a WAL gets used. That's a deliberate single point of truth: change a product's classification once, and every downstream calculation picks it up automatically, rather than needing to hunt down every place that might have hardcoded an assumption about how a given product behaves.

As a business decision — this is where funding strategy actually lives. Correctly separating “sticky” deposits from volatile ones tells us how much of our deposit book we can genuinely rely on as long-term funding — which in turn tells us how confidently we can fund longer-tenor, higher-margin lending without taking on liquidity risk we haven't priced for. Get this wrong in either direction and you either leave profitable lending capacity on the table, or fund it with a deposit base that isn't as stable as you assumed.

Decision: the Minimum Balance Method for persistence (four tranches, not a continuous curve)

Mathematically — we're estimating a distribution of how long balance actually persists, and reducing it to four discrete buckets (overnight/short/medium/long) rather than a continuous function. That's a real simplification, but a defensible one: it's coarse enough to calibrate reliably from observed data, and fine enough to separate the genuinely volatile portion of a balance from the genuinely core portion. Non-maturing liability modelling has its own dedicated academic literature for exactly this reason — Jarrow and Van Deventer (1998) built one of the earliest arbitrage-free frameworks for valuing demand deposits as a mix of a market-rate-linked component and a genuinely sticky core, and Kalkbrener and Willing (2004) extended that into a stochastic framework treating deposit volumes, deposit rates, and market rates as three separate risk factors rather than one.

Financially — this is how we put a number on deposit franchise value. The "core" tranches are the part of the deposit book we can treat as reliable, long-duration funding; the volatile tranche is the part we can't. Getting that split wrong in either direction either understates what our deposit franchise is actually worth to us, or overstates how much of it we can safely lend against.

In the software — we expand one account into up to four rows rather than trying to represent a distribution inline on a single row. That keeps every downstream consumer (rate lookup, charge calculation, consolidation) working with the same simple shape — one row, one WAL, one balance — regardless of whether the account behind it was ever split.

As a business decision — this is a direct lever for how aggressively we can price and grow long-tenor lending. A deposit franchise with a large, well-evidenced core tranche is a genuine competitive funding advantage; this method is how we turn "we believe our deposits are sticky" into a defensible, quantified input to that strategy rather than an assumption sitting outside the pricing model entirely.

Decision: prepayment adjustment for amortizing lending

Mathematically — weighted average life for any prepayable loan is, by definition, not computable from the amortization schedule alone; a prepayment assumption is a required input, not an optional refinement. There's no version of "just use the contractual schedule" that's actually correct here.

Financially — overstating a mortgage book's WAL means underpricing the refinancing risk embedded in it, which shows up later as unexpected duration mismatch if a large share of the book prepays faster than priced for.

In the software — we built a two-tier fallback (exact product match, then a broader class-level average) rather than a single hard lookup, so a newly added or reclassified product still gets priced sensibly instead of falling through to a contractual figure we already know overstates its real life.

As a business decision — this is what lets us price fixed-rate lending competitively without quietly taking on more duration risk than we think we have. Knowing the real, prepayment-adjusted funding requirement of a loan book is what makes it possible to decide how aggressively to grow that book in the first place.

Decision: a manual override, but only where it earns its place

Mathematically/financially — every behavioural model is an estimate, and every estimate can be wrong for a specific, identifiable reason a model can't see (a known one-off event, a data quality issue in a specific tenor class). An override mechanism is standard model-risk practice: regulatory guidance on model risk management explicitly calls for "effective challenge" of models — critical, documented analysis by informed parties able to identify model limitations and correct for them (Board of Governors of the Federal Reserve System & Office of the Comptroller of the Currency [SR 11-7], 2011) — models inform, but a documented, deliberate correction should still be possible.

In the software — we only wire the override into Persistence and Prepayment rows, never Contractual. For a contractual account, `CALCULATED_WAL` is already a passthrough of a real date; routing that through a second override mechanism would just duplicate the manual contractual-entry path one step earlier in the cascade, for no benefit.

As a business decision — every override is a deliberate, traceable correction rather than a silent one. That matters for governance: when ALCO or an auditor asks "why is this account priced the way it is," there's a clean answer either way — either the model's own logic, or a specific, recorded reason it was overridden.

See also `docs/behavioural_wal.md`, `docs/prepayment_wal.md`, `docs/persistence_structural_split_wal.md`.

References

Basel Committee on Banking Supervision. (2016). *Standards: Interest rate risk in the banking book* (BCBS368). Bank for International Settlements. https://www.bis.org/bcbs/publ/d368.pdf

Board of Governors of the Federal Reserve System, & Office of the Comptroller of the Currency. (2011). *Supervisory guidance on model risk management* (SR 11-7 / OCC Bulletin 2011-12). https://www.federalreserve.gov/boarddocs/srletters/2011/sr1107.pdf

Jarrow, R. A., & Van Deventer, D. R. (1998). The arbitrage-free valuation and hedging of demand deposits and credit card loans. *Journal of Banking and Finance, 22*(3), 249–272.

Kalkbrener, M., & Willing, J. (2004). Risk management of non-maturing liabilities. *Journal of Banking & Finance, 28*(7), 1547–1568.